# TrustPro, Full Build-Prompt Queue (Claude Code)

A complete, ordered queue of **atomic prompts**. Each prompt is **one focused task** sized for a single Claude Code session and a single PR, so the agent stays focused and you review small diffs. Run them top to bottom; each lists what it depends on by number.

**Platform:** TrustPro is **web-only**. It is a Next.js app, used in any browser and **installable as a PWA** on mobile or desktop. There is **no native mobile app** (no Expo / React Native). All references to "mobile" mean the responsive PWA on a phone browser.

**Source of truth:** the TrustPro SRS & System Architecture document. Requirement IDs (e.g. `TRU-02`, `PAY-01`) and the tier / Trust Score tables are authoritative; prompts reference them instead of restating them.

## STEP 0, one-time setup (paste into `CLAUDE.md`, do not run as a task)

This primes every session. Paste it into `CLAUDE.md` at the repo root once, then start the queue at PROMPT 01.

```
PROJECT: TrustPro, a trust-first local-services / skilled-labour marketplace for Cameroon (Douala and Yaoundé first).
WEB-ONLY: a Next.js App Router app, installable as a PWA on mobile/desktop, or used on the web. NO native mobile app.
The moat is TRUST, not listings: a continuously updated Trust Engine (identity, professional, marketplace, financial),
progressive verification (Tiers 0-3), portfolio evidence, transparent quotes, and escrow. Bilingual EN/FR. Currency XAF only.
Payments via Campay. Users are mobile-first (phone browser), low-bandwidth, non-technical.

MONOREPO (pnpm workspaces, Bun runtime):
  apps/web   Next.js App Router (TypeScript), run on Bun. Marketing site + installable PWA app. TailwindCSS + shadcn/ui
             (copied in and fully restyled to TMDS) + Anime.js v4 + Lenis + Apollo Client + next-intl + next-themes + React Hook Form + Zod.
             Maps via Mapbox GL JS. Location via the browser Geolocation API.
  apps/api   NestJS (TypeScript) on Bun. REST controllers + GraphQL resolvers + PostGIS. WebSocket gateways for real-time.
  packages/api-client        shared client config, GraphQL operations, REST mutation helpers.
  packages/database          Prisma client, migrations, PostGIS/OSM SQL views.
  packages/typescript-config shared tsconfig.
  Turborepo runs workspace tasks (turbo run dev/build/lint/type-check); Husky pre-commit; root scripts are the only entry
  point (dev, build, db:*, docker:*, env:sync, full-check). scripts/sync-env.mjs fans a single root .env out to apps.
  Local infra: docker-compose.yml with postgis/postgis (PostGIS preinstalled) + redis:7 for Docker Desktop.
  Redis roles: BullMQ queues (notifications, webhook retries), rate-limit store, WebSocket pub/sub adapter.

DUAL API RULE (do not violate):
  READS  -> GraphQL (NestJS resolvers -> Apollo Client normalized cache). Clients request exact fields.
  WRITES -> REST (NestJS controllers). All mutations, uploads, webhooks. Called via fetch/axios, bypassing the GraphQL link.
  Web streams GraphQL in RSC via @apollo/experimental-nextjs-app-support. Apollo cache is the read single-source-of-truth.

DATA: PostgreSQL + PostGIS (GIST indexes). Prisma ORM. Complex spatial ops via reviewed raw SQL / DB views, not Prisma fields.
  Index every frequent query path. No N+1: always select/include + pagination.

SPATIAL (Mapbox is presentation only, never authoritative for math):
  OSM (baseline) -> PostGIS (store/index + spatial math) -> CARTO (spatial SQL + dynamic MVT tiles) -> Mapbox GL JS (render).
  BigDataCloud for server-side reverse-geocoding. A human pin / landmark ALWAYS overrides auto-detected location.
  Real-time position uses the browser Geolocation API streamed over WebSocket while the relevant screen is open
  (web has no reliable background tracking; do foreground streaming with explicit consent).

AUTH & SECURITY: Auth.js with stateless JWT (access + refresh); secrets in a managed store. RBAC guards on every write and
  privileged read. One identity holds multiple roles (Worker, Client, Business owner, Agent, Admin); tier + Trust Score attach
  to the IDENTITY, not the role, so a role switch never re-verifies. Validate all input (Zod/DTO) before the DB. Rate-limit
  public + auth/OTP endpoints. Honeypots on public forms. KYC media (ID, selfie) is access-controlled + encrypted at rest.

NOTIFICATIONS & OTP (DISPATCHER PATTERN, LAUNCH CONSTRAINT): controllers EMIT typed domain events (@nestjs/event-emitter),
  they never call a provider directly. A NotificationDispatcher applies per-user preferences (channel x category; security
  notices bypass), selects channels by configurable priority, and delegates to providers behind a NotificationChannel
  interface: WebPush (VAPID, multi-device registry with expiry cleanup), branded Email, and a stubbed SMS provider
  (feature flag OFF; SMS is DEFERRED, paid API later). Every attempt is written to a NotificationLog (channel, provider,
  recipient, templateKey, status, error). Templates live in one localized EN/FR store keyed by templateKey, never hardcoded.
  Delivery is queued + retried via BullMQ on Redis. Phone numbers are normalised to E.164 at capture so SMS flips on later
  with zero call-site changes. OTP codes go through this same channel service (email/console at launch).

PAYMENTS & ESCROW: Campay for Mobile Money (MTN, Orange), card, bank in XAF (collection + payout). Escrow is a server-side
  state machine (Paid -> Accepted -> In Progress -> Completed -> Released; plus Refunded / Disputed). ~10% fee shown before pay.
  Every transition idempotent + reconciled against Campay webhooks (keyed by external ref). Audit-log every money/verification event.

STORAGE: AWS S3 (or compatible) for portfolio media, KYC docs, voice notes. Uploads via REST multipart with validation + signed access.

DESIGN SYSTEM (TMDS, mandatory, tokens only):
  Colours Indigo #3D5AFE, Success #14B86A, Accent #00C2A8, Warning #F59E0B, Danger #EF4444, neutral 50-900. Type: Geist/Inter Tight
  headings, Inter body, IBM Plex Sans numbers. 8pt spacing. Radius cards 20 / inputs 16 / buttons 14 / images 24. Soft shadows.
  Light + dark. Style "Modern Soft Premium". Motion via Anime.js v4: explains change, transform/opacity only, 60fps, respect
  prefers-reduced-motion, skeletons over spinners. Marketing = animated profile; app/admin = calm; admin dark/dense/chart-heavy.

ACCESS MODEL: The guest first-interface lets UNAUTHENTICATED visitors fully browse and explore the platform (search workers, browse posted jobs,
open full worker profiles, basic exploration) with NO account, to showcase the platform's potential and drive sign-up; once authenticated, users
see a ROLE-BASED home for their active role (Worker/Client/Business), not the guest view. Every state-changing action
  (post/apply/contact/pay/save/review) requires login, then RESUMES the original intent. The marketing landing page is separate
  from the installable app shell and is EXCLUDED from the service-worker scope (fast, crawlable, SEO-first).

ENGINEERING STANDARDS (every prompt): central structured JSON logger (level, scope, requestId, durationMs, sanitised summary, no
  secrets); requestId generated in the first middleware, in every log line, returned in error responses; log info (req in/out, DB done),
  warn (slow query >300ms), error (DB/payment/unhandled); app/global-error.tsx from the start; every mutating form has inline
  success/error + field validation, no silent failures; accurate auth errors (credentials vs session); migrations run at deploy only
  (prisma migrate deploy in CI before boot; migration_lock.toml committed; .sql UTF-8 no BOM); public pages static/ISR; SEO on
  marketing; full EN/FR, no hardcoded strings; CI gate = typecheck + lint + tests + prisma validate + clean high/critical security scan.

DEFINITION OF DONE (every PR): typechecks; lints; tests for new logic; reads via GraphQL/Apollo and writes via REST/fetch; RBAC
```

```
  enforced; inputs validated; indexes added for new query paths; loading/empty/error states; EN/FR strings; a11y (keyboard, focus,
  contrast, reduced-motion); requestId logging; no secrets committed; PR references the SRS requirement IDs + the PROMPT number.

CONVENTIONS: TypeScript everywhere. One NestJS module per SRS module. Shared types + GraphQL ops + Prisma client in packages/*.
  Brand token: "TrustPro". Add each feature's Prisma models + migration inside the prompt that introduces the feature (incremental schema).
```

---

## Model routing and saving your Pro limits (read this once, carefully)

**First, the thing that confused you: a “session” is NOT the 5-hour window.** They are two different things:

- The **5-hour window** is a USAGE BUDGET. It is a rolling meter of how much compute you have used. Every message you send (in Claude Code or Claude chat) adds to it; when it fills, you wait for it to roll over. There is also a weekly cap on top.
- A **session** is just ONE CONVERSATION inside Claude Code: the chat history the model can currently see. `/clear` wipes that history and gives you an empty conversation. It costs nothing, takes one second, and you keep working immediately in the same terminal, inside the same 5-hour window.

So NO, you do not spend one 5-hour window per prompt. The workflow is: paste a prompt, let it finish, review, commit, type `/clear`, paste the next prompt right away. You will typically get through SEVERAL prompts per 5-hour window (small Haiku prompts: many; big opusplan prompts: fewer). `/clear` is not a “cost”, it is the thing that SAVES you budget.

**Why `/clear` between prompts saves so much:** the model is stateless; every single message you send re-uploads the ENTIRE conversation so far (all your prompts, all its replies, every file it read). If you run prompt after prompt in one long conversation, prompt 10 is silently re-paying for prompts 1-9 on every message. Clearing after each prompt means each prompt only ever pays for itself plus CLAUDE.md. This one habit saves more budget than any model choice.

**Your loop for every prompt in this queue:** 1. Pick the next prompt; set the model per its tag (`/model haiku`, `/model sonnet`, or `/model opusplan`). 2. Paste the prompt block. Let it work; answer its questions briefly. 3. If ONE prompt runs long and the context gets heavy mid-task, use `/compact` (it summarises the conversation in place so you can continue the SAME task with a smaller context). `/compact` = keep working on this task; `/clear` = task finished, wipe everything. 4. Review the diff, run the Done checks, commit. 5. `/clear`. Next prompt. (`/clear` does NOT reset the 5-hour meter; nothing does except time.)

**The other levers, in order of impact:** - **Keep CLAUDE.md lean.** It is injected into EVERY message of EVERY session, so every extra line is paid for constantly. The CLAUDE.md provided with this project is already trimmed; resist appending to it. - **Model tags.** All models drain the same budget, but at very different rates: Haiku is roughly a third of Sonnet per token; Opus is several times Sonnet. `opusplan` plans with Opus then executes with Sonnet, so you get deep reasoning without Opus-priced execution. Follow the tag on each prompt; return to `/model sonnet` after an opusplan prompt. - **Don’t trigger extended thinking on simple prompts.** Words like “think hard” or “ultrathink” make the model emit thinking tokens, billed as output. Reserve them for opusplan-tagged prompts, if at all. - **Point, don’t let it wander.** Name exact files in follow-ups (“edit apps/api/src/trust/trust.service.ts”) instead of letting it read 15 files to find one. Every file it reads enters the context and is re-paid on every later message of that session. - **Watch `/cost` and `/context`** occasionally. If context is mostly old file dumps, you waited too long to `/clear`. - **If you hit the 5-hour cap mid-prompt:** commit what is done, note where you stopped, wait for the window to roll, `/clear`, and re-run the same prompt with a one-line note of what is already complete.

Every prompt below carries a suggested **Model** tag: **Haiku** (mechanical: scaffolding, tokens, schema-only, config, static UI), **Sonnet** (default feature work), **opusplan** (money, security, state machines, spatial, sync).

# THE QUEUE

> Phases: **0** Foundation · **1** Access & Roles · **2** Trust Engine · **3** Discovery & Profiles · **4** Transactions · **5** Messaging, Safety & Notifications · **6** Reviews, Business & Admin · **7** PWA Hardening & Marketing.

## Phase 0: Foundation

**Model: Sonnet**

```
PROMPT 01: Initialize the monorepo and runtimes
Depends: none. Covers: SRS §8.
Create a pnpm-workspace.yaml monorepo with apps/web (Next.js App Router, TypeScript, run on Bun) and apps/api (NestJS, TypeScript,
run on Bun), plus packages/typescript-config (shared tsconfig base). Add shared ESLint + Prettier and strict TypeScript. Add a root
README documenting the Bun runtime and the dual-API rule. Do NOT create any mobile app; this is web-only.
Done: bun dev boots web and api locally; both apps extend the shared tsconfig; lint + typecheck run at the root.
```

**Model: Sonnet**

```
PROMPT 01.1: Root scripts, env sync, husky, keygen
Depends: 01. Covers: SRS §24.2-24.3.
Create the canonical root package.json scripts (the ONLY entry points; no app-level invocation): prepare (husky + lint-staged),
env:sync (scripts/sync-env.mjs fanning the root .env to apps), dev / dev:web / dev:api (env:sync first; dev:api builds
packages/database first; turbo run dev --parallel), build / build:web / build:api, lint, lint:i18n (ESLint pass forbidding
hardcoded user-facing strings), type-check, test, format, full-check (lint + lint:i18n + type-check + test),
db:generate / db:push / db:studio / db:migrate / db:migrate:deploy / db:migrate:reset / db:seed (delegating to
packages/database), db:up / db:down / db:reset, docker:up / docker:reset / docker:logs, dev:setup (build database pkg ->
db:up -> generate -> migrate -> seed), backup:db (scripts/docker-db-backup.sh), keys (scripts/keygen.cjs generating JWT +
VAPID secrets), 32-bytes, clean. Add Turborepo config (turbo.json) for dev/build/lint/type-check with caching, Husky
pre-commit running lint-staged, and a root .env.example documenting every variable (DATABASE_URL, DIRECT_URL, REDIS_URL,
JWT_*, VAPID_*, CAMPAY_*, S3_*, MAPBOX_TOKEN, CARTO_*, BIGDATACLOUD_KEY, RESEND_API_KEY).
Done: pnpm dev boots both apps via turbo after env:sync; full-check runs all gates; keys generates secrets; .env.example is complete; husky
blocks a commit that fails lint-staged.
```

**Model: Haiku** (compose file + scripts)

```
PROMPT 01.2: Docker Compose for local Postgres (PostGIS) + Redis
```

```
Depends: 01.1. Covers: SRS §24.1.
Create docker-compose.yml at the repo root for Docker Desktop: service postgres using the postgis/postgis image (PostGIS
preinstalled) on port 5432 with POSTGRES_DB=trustpro, a named volume, and a pg_isready healthcheck; service redis using
redis:7-alpine on port 6379 with appendonly persistence and a healthcheck. Wire the db:up / db:down / db:reset / docker:*
root scripts to it and point the local .env DATABASE_URL/REDIS_URL at these containers. Add scripts/docker-db-backup.sh
(pg_dump into ./backups with a timestamp).
Done: pnpm db:up starts healthy postgres+redis; prisma migrate dev connects; pnpm backup:db writes a dump; db:reset recreates cleanly.
```

**Model: Sonnet**

```
PROMPT 02: CI/CD pipeline and gates
Depends: 01. Covers: engineering standards, SRS §18.1.
Add GitHub Actions: install, typecheck, lint, test, prisma validate, and a high/critical security-scan gate that blocks merge.
Add a deploy job that runs `prisma migrate deploy` BEFORE the server boots. Ensure migration_lock.toml is committed with provider=postgresql.
Done: CI runs on PRs and blocks on any failed gate; the deploy job order runs migrations before boot.
```

**Model: Sonnet**

```
PROMPT 02.1: GitHub Actions workflows (deploy, path-triggered migrate, manual seed)
Depends: 02, 05. Covers: SRS §25.
Create three workflows adapted from the proven template:
(1) .github/workflows/deploy-production.yml: on push/PR to main. Job "validate": pnpm install --frozen-lockfile ->
db:generate -> type-check -> lint. Job "db-migrate" (main only, and only when the commit touches packages/database/prisma):
prisma migrate deploy against DIRECT_URL, then a schema-drift gate `prisma migrate diff --from-url $DIRECT_URL
--to-schema-datamodel packages/database/prisma/schema.prisma --exit-code` that fails the pipeline on drift. Job "summary"
(if: always()): writes a step summary noting validation + migration results and that the API auto-deploys via Railway branch
integration and web via Vercel Git integration.
(2) .github/workflows/prisma-migrate.yml: path-triggered on packages/database/prisma/** pushes to main; migrate deploy + the
same drift verification.
(3) .github/workflows/manual-seed.yml: workflow_dispatch with a "confirmation" input; runs only if the input equals CONFIRM;
db:generate then db:seed against production secrets.
All DB steps use secrets.DIRECT_DATABASE_URL || secrets.DIRECT_URL. pnpm version pinned; node 20 with pnpm cache.
Done: the three workflows are valid; migrate jobs are gated to main + prisma paths; the drift check fails on an out-of-sync schema; manual seed
refuses without CONFIRM.
```

**Model: Haiku** (scoped utility)

```
PROMPT 03: Structured logger
Depends: 01. Covers: SRS §19.
Add a central logger (apps/api/src/lib/logger and a thin web equivalent): JSON in prod, pretty in dev, with fields level, scope, msg,
requestId, durationMs, and a sanitised payload/result summary. Never log secrets, tokens, or full KYC data.
Done: logger emits structured lines in both apps; a sample info/warn/error render correctly; no secret leakage in output.
```

**Model: Sonnet**

```
PROMPT 04: Request-ID, error contract, and error boundaries
Depends: 03. Covers: SRS §19, engineering standards.
Add request-ID middleware in apps/api (first in the chain): generate a uuid, attach to request context, include in every log line, and
return it in all error responses plus an x-request-id header. Define a typed error contract { error: { code, message, requestId, fields? } }
used by every REST controller, with a matching GraphQL error formatter and a NestJS global exception filter. Add app/global-error.tsx and
route-level error boundaries in web, with a client helper that surfaces message + requestId in inline form feedback.
Done: a forced error returns the typed contract with a requestId that matches the server log; web shows a friendly error boundary.
```

**Model: opusplan** (data architecture + PostGIS)

```
PROMPT 05: Prisma + PostGIS baseline and identity models
Depends: 02. Covers: SRS §6, §11.
In packages/database, configure Prisma against PostgreSQL and enable the PostGIS extension via the initial migration. Add the User and
UserRole models (roles: WORKER, CLIENT, BUSINESS, AGENT, ADMIN) and an AuditLog model. Emit the shared Prisma client and export it for
web + api. Add a seed skeleton. Establish the pattern that later prompts add their own models + migrations.
Done: migrate deploy creates the schema with PostGIS enabled; the shared client imports cleanly in apps/api; seed runs.
```

**Model: Sonnet**

```
PROMPT 05.1: Redis + BullMQ foundation
Depends: 05, 01.2. Covers: SRS §17, §24.1.
Wire Redis into apps/api: a RedisModule providing a shared ioredis connection from REDIS_URL; a BullMQ setup with named
queues "notifications" and "webhooks" (default retry/backoff policy, dead-letter handling, a processor registration pattern);
the rate-limit guard switched to the Redis store; and the WebSocket gateway adapter backed by Redis pub/sub so gateways can
scale horizontally later. Health check pings Redis.
Done: /health reports Redis; a test job enqueued to "notifications" is processed with retry on thrown error; rate limiting
counts survive an API restart; two API instances share WS broadcasts through the adapter.
```

**Model: Haiku** (tokens only)

```
PROMPT 06: TMDS tokens, Tailwind theme, light/dark
Depends: 01. Covers: SRS §20.1-20.3.
In apps/web, define TMDS design tokens as CSS variables + a Tailwind theme: colours (Indigo #3D5AFE, Success #14B86A, Accent #00C2A8,
Warning #F59E0B, Danger #EF4444, neutral 50-900), typography (Geist/Inter Tight headings, Inter body, IBM Plex Sans numbers), 8pt spacing,
radii (cards 20 / inputs 16 / buttons 14 / images 24), soft shadow scale. Wire light + dark themes via next-themes. No hardcoded values
anywhere.
Done: toggling the theme swaps tokens app-wide; a token reference renders in both themes; nothing is hardcoded.
```

**Model: Haiku** (restyling primitives)

```
PROMPT 07: Restyle shadcn primitives to TMDS
Depends: 06. Covers: SRS §20.5.
Copy the shadcn/ui primitives into the workspace and fully restyle them to TMDS (button, input, select, dialog, drawer, tabs, toast,
skeleton, card, dropdown, tooltip). They must not look like default shadcn.
Done: each primitive renders in the TMDS look in both themes and uses only tokens.
```

**Model: Sonnet**

```
PROMPT 08: Motion system and experience profiles
Depends: 06. Covers: SRS §20.4, §9.4.
Build an Anime.js v4 motion wrapper with presets (page transition ~450ms fade+scale+blur, card hover lift+scale 1.02, button
micro-interaction, checkmark draw, count-up). All motion uses transform/opacity only and respects prefers-reduced-motion. Add Lenis smooth
scroll scoped to marketing routes. Provide two layout wrappers: "marketing" (strong staggered reveals) and "app" (minimal motion).
Done: presets animate at 60fps; reduced-motion disables non-essential motion; the two profiles are selectable per route group.
```

**Model: Haiku** (static styleguide)

```
PROMPT 09: Styleguide route
Depends: 07, 08. Covers: SRS §20.
Add a /styleguide route rendering every token, restyled primitive, and motion preset with a light/dark toggle, for visual QA.
Done: /styleguide shows the full system in both themes and each motion preset can be triggered.
```

---

## Phase 1: Access & Roles

**Model: Sonnet**

```
PROMPT 10: Accounts and sign-up
Depends: 05. Covers: ACC-04.
In a NestJS auth module, build sign-up (REST write): capture role intent ("find_work" | "hire") and phone number; create User + initial
UserRole. Normalise and store the phone number in E.164 (+237...) at capture even though SMS is deferred. Validate with Zod/DTO; rate-limit the
endpoint.
Done: a POST creates a User with the chosen initial role and a stored phone number; invalid input returns the typed error contract.
```

**Model: opusplan** (abstraction must be right for SMS-later)

```
PROMPT 11: Notification dispatcher core (channel interface, log, localized templates)
Depends: 04, 05.1. Covers: notification/OTP constraint, NOT-02, SRS §17, §26.1.
Build the dispatcher core exactly per the dispatcher pattern in CLAUDE.md:
- NotificationChannel interface with providers: Email (active, branded templates), WebPush (active, fully wired later), a
  stubbed SMS provider registered but feature-flagged OFF, and a dev/console provider for local OTP.
- NotificationDispatcher subscribing to typed domain events via @nestjs/event-emitter; controllers EMIT events and never call
  a provider. The dispatcher applies per-user preferences and channel priority, and enqueues delivery on the BullMQ
  "notifications" queue with retry.
- Prisma models + migration: NotificationLog (userId?, channel, provider, recipient, templateKey, subject?, status
  PENDING/SENT/FAILED, errorMessage?, metadata, createdAt, sentAt?; indexes [userId,channel,createdAt], [status,createdAt],
  [templateKey,status]) and NotificationPreference (userId, channel, category, enabled; unique [userId,channel,category]).
  Seed default preferences at user creation. Security-critical templates (password reset, new-device login) bypass
  preferences and always use email.
- A central localized template store keyed by templateKey with EN/FR variants per channel (email subject+HTML, push
  title/body, sms text). Never hardcode content at dispatch sites; log the templateKey, not rendered content.
Done: emitting a test event dispatches via email (console in dev), writes a NotificationLog row for every attempt including
failures, respects a disabled preference, and enabling the SMS flag routes to the stub with zero call-site changes.
```

**Model: Sonnet**

```
PROMPT 12: Phone OTP verification
Depends: 10, 11. Covers: ACC-04.
Add OTP issue + verify (REST) using the NotificationChannel service to deliver a 6-digit code via the active channel (email at launch, or a
dev/console provider). Resend with countdown, expiry, and attempt throttling. Do not call SMS directly anywhere.
Done: a user requests a code, receives it via the active channel, and verifies; throttling and expiry work; switching to SMS later needs no code
change here.
```

**Model: opusplan** (auth, security-critical)

```
PROMPT 13: JWT sessions and login
Depends: 12. Covers: ACC-05, ACC-09.
Wire Auth.js with stateless JWT (access + refresh, secure rotation). Add login, logout, and refresh. Auth errors must be accurate:
"Invalid phone/email or password" for bad credentials, "Session expired" only on real expiry.
Done: a verified user logs in and the session persists across reloads and refresh; auth errors are accurate.
```

**Model: Sonnet**

```
PROMPT 14: Password reset and 2FA scaffold
Depends: 13. Covers: ACC-08.
Add forgot-password, reset-password (delivered via the NotificationChannel service), and an optional 2FA scaffold that can be toggled per user.
Done: a user resets a password via the active channel and can enable/disable 2FA.
```

**Model: opusplan** (RBAC + identity, security-critical)

```
PROMPT 15: Multi-role identity and RBAC
Depends: 13. Covers: ACC-06, RBAC.
Allow one User to hold many UserRole. Attach tier + Trust Score to the identity, not the role. Add RBAC guards enforcing role/permission on
every write and privileged read, driven by an activeRole JWT claim. Add a GraphQL `me` query returning identity + roles + activeRole + tier +
```

```
trustScore.
Done: `me` returns the identity graph; the server rejects actions not permitted for the active role.
```

**Model: Sonnet**

```
PROMPT 16: Role Switcher
Depends: 15. Covers: ACC-07.
Add a first-class Role Switcher in primary navigation (top bar on desktop, accessible control on the phone-width PWA) that updates the
activeRole claim and swaps the available experience instantly with an animated transition, persisting the choice. Never re-authenticate on
switch.
Done: one account toggles Worker <-> Client without re-login; the UI and permitted actions update immediately; the choice persists.
```

**Model: Sonnet**

```
PROMPT 17: Auth UI screens
Depends: 13. Covers: ACC-11.
Build the auth screens in apps/web inside the standard shell (never naked): split-screen on desktop (form left, brand panel right), full-screen
on phone width. Include the OTP screen with resend countdown, inline field-level validation, success/error banners, and EN/FR.
Done: sign-up, OTP, login, and reset screens work end to end with inline feedback in both languages.
```

**Model: Sonnet**

```
PROMPT 18: Installable PWA app shell
Depends: 06. Covers: SYS-01.
Give the app a manifest + service worker so the app shell is installable on phone/desktop, with app-shell caching. Keep this scoped to the
APP only; the marketing landing (P59) must be excluded from the service-worker scope.
Done: the app is installable and passes an install audit; the service worker controls only app routes, not the landing page.
```

**Model: Sonnet**

```
PROMPT 19: Guest access and gated-action pattern
Depends: 15, 18. Covers: ACC-01, ACC-02.
Implement guest browsing (no account) for discovery, and a reusable requireAuth(intent) wrapper: any state-changing action taken by a guest
(post/apply/contact/pay/save/review) captures the intent, routes to login/register, and on success REPLAYS the original intent (e.g. re-opens
the drafted job).
Done: a guest can browse; a gated action bounces to auth and resumes exactly where it left off afterward.
```

**Model: Sonnet**

```
PROMPT 20: Worker onboarding (gamified)
Depends: 16. Covers: ACC-10, TRU-05.
Build the Worker onboarding flow: services, starting prices, availability, portfolio upload, ID upload, selfie, location. Show a
profile-completion percentage with unlockable-benefit messaging (not a raw checklist). Autosave each step; allow resume. Media uploads go to
S3 via REST multipart with progress + validation. Celebratory unlock (confetti) only on completion.
Done: a worker completes onboarding with autosave/resume; completion percentage updates live; uploads land in S3 with validation; EN/FR
throughout.
```

**Model: Haiku** (simple form)

```
PROMPT 21: Client onboarding
Depends: 16. Covers: ACC-10.
Build the Client onboarding flow: basic profile + location, with the same completion-percentage framing, autosave/resume, and EN/FR.
Done: a client completes onboarding with autosave; location is captured and stored.
```

**Model: Sonnet**

```
PROMPT 22: Business onboarding
Depends: 16. Covers: ACC-10, BIZ-04 (intake only).
Build the Business onboarding flow: company details, RCCM/registration + document upload, and initial team setup, with the completion-percentage
framing, autosave/resume, and EN/FR. (Approval/verification of the business is handled in Trust/Admin phases.)
Done: a business completes intake with documents uploaded to S3 and an initial team recorded.
```

---

## Phase 2: Trust Engine

**Model: Sonnet**

```
PROMPT 23: Trust schema
Depends: 05. Covers: SRS §6, §21.
Add Prisma models + migration: TrustScore (subject, four sub-scores, total), TrustEvent (append-only: type, delta, actor, ts),
VerificationRecord (checkType, status pending/verified/rejected, reviewer, reason, ts), Document (kind, s3Key, status, visibility),
Endorsement (author, relationship, quote, rating), and a TierConfig table for admin-tunable thresholds.
Done: migration applies; models are exported from the shared client; TrustEvent is append-only by convention.
```

**Model: opusplan** (core scoring logic, get it exact)

```
PROMPT 24: Trust Score service (point model)
Depends: 23. Covers: TRU-01, §21.5.
Implement TrustScoreService: recompute a 0-100 score from the TrustEvent log using the exact point model in SRS §21.5 (start 15; phone +10;
ID +20; field +15; portfolio +10; completed jobs +15; repeat customers +10; excellent reviews +15; disputes -20; cancelled -10; fake docs =>
permanent suspension overriding score). Produce the four sub-scores. Pure + idempotent; clamp [0,100]. Unit-test each rule.
Done: a given event sequence yields a deterministic score + sub-scores; the suspension rule overrides the numeric score.
```

**Model: opusplan** (tier-gate logic)

```
PROMPT 25: Tier derivation service
Depends: 24. Covers: TRU-02, §21.4.
Implement tier derivation (Tiers 0-3) from the gates in SRS §21.4 (Basic; Phone+ID submitted; ID approved+portfolio; Field verified +>=20 jobs
+>=4.8 rating), reading thresholds from TierConfig (admin-tunable, not literals). Tier is derived from score + completed checks, never set
manually.
Done: an identity lands in the correct tier at each gate; changing TierConfig moves the boundary; tier cannot be set by hand.
```

**Model: Sonnet**

```
PROMPT 26: Trust reads and change events
Depends: 25. Covers: TRU-01, TRU-04.
Add GraphQL reads: trustScore(subjectId), trustBreakdown(subjectId) (per sub-score + inputs + how to improve), tier(subjectId). Emit a domain
event on tier change and on score-threshold crossings (consumed by badges + notifications). Feed Trust Score into default search ranking (used
in Phase 3).
Done: the reads return correct data; a tier change emits an event other modules can subscribe to.
```

**Model: opusplan** (KYC security + encryption)

```
PROMPT 27: Document upload and secure storage
Depends: 23, 20. Covers: TRU-06 (storage), NFR-SEC.
Build document upload (National ID front/back, selfie, proof of address, certificates, portfolio) to S3 via REST multipart, access-controlled
and encrypted at rest, visible only to the owner and authorised agents/admins. Signed, expiring access URLs. Validate type/size.
Done: an owner uploads KYC docs that are retrievable only by the owner + authorised roles; unauthorised access is denied.
```

**Model: Sonnet**

```
PROMPT 28: Verification records and friendly retries
Depends: 27, 26. Covers: TRU-06, TRU-07, TRU-08.
Create VerificationRecords per check with pending/verified/rejected states, timestamps, reviewer, and reason, plus re-upload on rejection.
Approvals/rejections emit TrustEvents (feeding P24). If automated ID OCR fails, return an actionable non-judgemental message
("We couldn't read your ID clearly. Try again in better lighting"). Confirmed fake docs trigger permanent suspension overriding the score.
Done: submitting docs creates pending records; approval bumps score + tier; rejection returns a friendly retry; fake docs suspend.
```

**Model: Sonnet**

```
PROMPT 29: Verification pipeline (agent/admin backend)
Depends: 28. Covers: AGT-02, AGT-03, ADM-03.
Model the review pipeline: Submitted -> Under Review -> Field Check -> Pending Approval -> Approved, with per-stage counts, average review time,
and approval rate. Add actions approve / reject (with reason) / request-more-info, RBAC-gated to agent/admin.
Done: an agent/admin moves a submission through every stage; metrics update; each action is RBAC-guarded and audit-logged.
```

**Model: opusplan** (capped/tapering algorithm)

```
PROMPT 30: Endorsements (capped, tapering)
Depends: 24. Covers: TRU-10.
Implement community endorsements (previous customer, workshop owner, quarter chief, landlord, community leader): author, relationship, quote,
rating. They add a modest, CAPPED amount to the Trust Score that TAPERS as verified job history accumulates. Implement the taper explicitly.
Done: endorsements raise the score within the cap and their influence shrinks as real job history grows.
```

**Model: Sonnet**

```
PROMPT 31: Founding Professional intake
Depends: 28. Covers: TRU-09, AGT-01.
Build a lightweight agent/admin intake tool to register a worker verified in person and grant an immediate "Founding Verified Professional"
badge, recording the manual field-verification TrustEvent.
Done: an agent field-verifies + badges a worker in a few steps; the score/tier reflect the field-verification event.
```

**Model: Sonnet**

```
PROMPT 32: Badges, milestones, and trust history
Depends: 26. Covers: TRU-11.
Add milestone badges (Founding Professional, tier upgrades, jobs milestones) triggered by trust events, and a trust-history timeline for the
owner. Emit notification events on unlock.
Done: crossing a milestone unlocks a badge and appends to the trust history; the owner can view the timeline.
```

**Model: Sonnet**

```
PROMPT 33: Trust UI components (ring + badge)
Depends: 26, 08. Covers: TRU-01, §20.
Build the TrustScoreRing (animated gauge + count-up) and TierBadge TMDS components, plus a "Basic Profile" label variant for unverified (Tier 0)
profiles. Never imply verification for an unverified profile.
Done: the ring animates to a score; the badge shows the tier; Tier 0 renders the explicit Basic Profile label.
```

**Model: Sonnet**

```
PROMPT 34: Trust Score breakdown screen
Depends: 33. Covers: TRU-03, TRU-04.
Build the Trust Score breakdown screen: each sub-score, its inputs, and concrete steps to improve it, reading from trustBreakdown.
Done: a user sees their four sub-scores with explanations and next actions to raise each.
```

**Model: Sonnet**

```
PROMPT 35: Gamified verification screen (worker)
Depends: 28, 33. Covers: TRU-05.
Build the worker verification screen as gamified progress: completion percentage, per-check status, and benefit unlocks that animate as checks
pass.
Done: a worker sees verification as progress with animated unlocks, not a bureaucratic checklist.
```

**Model: Sonnet**

```
PROMPT 36: Verification pipeline board (agent/admin UI)
Depends: 29. Covers: AGT-02, ADM-03.
Build the agent/admin pipeline board (calm, dense profile) with columns per stage, a secure document viewer, and approve / reject / request-
more-info
actions with reason capture.
Done: an agent/admin works the queue visually, views documents securely, and records decisions with reasons.
```

---

## Phase 3: Discovery & Profiles

**Model: Haiku** (schema only)

```
PROMPT 37: Catalog schema
Depends: 05. Covers: SRS §6, PRO-05.
Add Prisma models + migration: ServiceCategory (name en/fr, icon, workerCount), Service (workerId, categoryId, name, description, startingPrice,
isActive), PortfolioItem (type photo/video/project, s3Key, caption, group), Availability (dayOfWeek, startTime, endTime, exceptions).
Done: migration applies; models exported; a worker can be associated with services, portfolio, and availability.
```

**Model: opusplan** (PostGIS raw SQL + ingest)

```
PROMPT 38: Spatial schema and OSM ingest
Depends: 05. Covers: SRS §12.
Add GeoRegion (name, boundary geography, parentId) and geometry/geography columns on WorkerProfile.location and Job.location via raw SQL
migrations, each with a GIST index. Ingest an OSM extract for Douala + Yaounde into PostGIS (boundaries, roads, POIs) and seed GeoRegions.
Expose spatial reads through reviewed SQL views, not Prisma scalar fields.
Done: PostGIS holds OSM data with GIST indexes; GeoRegions are seeded; spatial columns are queryable via views.
```

**Model: Sonnet**

```
PROMPT 39: Reverse-geocoding service
Depends: 38. Covers: SRS §12, DIS-09.
Wrap BigDataCloud server-side reverse-geocoding in a geo service that resolves coordinates to an administrative area, caches results, and stores
locations with a source enum (device_gps | human_input | reverse_geocode).
Done: given coordinates, the service returns the administrative area and persists the source; repeat lookups hit the cache.
```

**Model: opusplan** (spatial math (KNN))

```
PROMPT 40: Proximity and nearby queries
Depends: 38. Covers: DIS-06, DIS-07 (distance).
Implement PostGIS proximity/nearby queries (KNN via GIST, distance, within-region) behind SQL views, exposed through the geo module. All spatial
math is server-side; never on the client.
Done: a "nearby workers" query returns distance-ranked results from PostGIS for a given point + radius.
```

**Model: Sonnet**

```
PROMPT 41: CARTO vector tiles
Depends: 38. Covers: SRS §12.
Integrate CARTO to serve dynamic MVT tiles from PostGIS (nearby-worker density, region layers) for map rendering. Mapbox GL JS will consume
these; Mapbox is presentation only.
Done: a map can request CARTO-served MVT tiles reflecting live PostGIS data.
```

**Model: Sonnet**

```
PROMPT 42: Location setter (current location + map pin)
Depends: 39, 41. Covers: DIS-09 (meeting notes: location setter button).
Build the Location Setter used across discovery: a quick-tap "Use my current location" (browser Geolocation API) AND an "open map to drop/adjust
a
pin" mode (Mapbox GL JS). A manual pin or landmark note ALWAYS overrides auto-detected location (store source=human_input). The chosen location
drives feed + search.
Done: one tap sets current location; the map lets the user drop/adjust a pin; a manual pin overrides GPS; the value drives downstream queries.
```

**Model: Haiku** (CRUD)

```
PROMPT 43: Service-category taxonomy CRUD (admin)
Depends: 37, 15. Covers: ADM-02, DIS-05.
Build admin CRUD for the ServiceCategory taxonomy that search and job posting are scoped to (RBAC: admin). Reads via GraphQL, writes via REST.
Done: an admin creates/edits/retires categories; the taxonomy is queryable for search + posting.
```

**Model: opusplan** (fuzzy search + ranking)

```
PROMPT 44: Search backend (fuzzy, scoped, no-provider fallback)
Depends: 43, 40, 26. Covers: DIS-04, DIS-05, DIS-06.
Build worker search (GraphQL read): fuzzy matching (pg_trgm) scoped to the admin taxonomy; incorporate proximity (P40) and Trust Score/tier
ranking
(P26). If no worker offers the requested service in the area, return an explicit "no provider yet in your area" signal, not an empty list.
```

```
Done: fuzzy queries return ranked, taxonomy-scoped results; the no-provider case returns the friendly signal.
```

**Model: Sonnet**

```
PROMPT 45: Filters backend
Depends: 44. Covers: DIS-07, DIS-08.
Add filters to search: proximity radius, category, price range, minimum Trust Score/tier, rating, availability, verified-only, response time.
Ranges are DERIVED from data (never hardcoded). Higher-tier / higher-score workers are eligible for priority placement.
Done: every filter narrows results correctly; ranges come from data; ranking honours tier/score.
```

**Model: Sonnet**

```
PROMPT 46: Worker profile read
Depends: 37, 26. Covers: PRO-01, PRO-02, PRO-03.
Build the worker profile GraphQL read: identity, animated Trust Score inputs, tier, headline stats (jobs completed, rating, on-time %,
verification),
services (transparent pricing), portfolio (photos/videos/projects, before/after), availability, certifications, endorsements, reviews summary.
Done: the query returns everything the profile screen needs in one efficient shape (no N+1).
```

**Model: Sonnet**

```
PROMPT 47: Business profile read
Depends: 22, 26. Covers: PRO-06.
Build the business profile read: logo, description, contact, RCCM, member-since, aggregate stats, and tabs data
(About/Jobs/Reviews/Team/Documents).
Done: the query returns the business profile graph efficiently.
```

**Model: Sonnet**

```
PROMPT 48: Profile management (owner edits + media)
Depends: 46. Covers: PRO-05, PRO-07.
Build owner edit flows (REST writes): manage Services (name, category, description, starting price, active), portfolio items (add/remove/group,
before/after), and weekly Availability (slots + exceptions). Media to S3 with progress + validation. Inline success/error feedback.
Done: a worker edits services/portfolio/availability with inline feedback; media uploads validate and persist.
```

**Model: Sonnet**

```
PROMPT 49: Worker Card component
Depends: 33, 45. Covers: DIS-02.
Build the Worker Card (TMDS): photo, verified badge, animated Trust Score ring, portfolio preview, price-from, distance, availability, primary
action.
It expands on hover/tap to reveal more portfolio + rating.
Done: the card renders all signals and expands smoothly; it is reused across discovery surfaces.
```

**Model: Sonnet**

```
PROMPT 50: Worker profile screen
Depends: 46, 49. Covers: PRO-01, PRO-04.
Build the public worker profile screen (Apple-quality, TMDS): cover, photo, Trust Score ring, tier badge, headline stats, tabs
About/Services/Portfolio/Reviews (portfolio first-class with before/after), transparent pricing, availability calendar. Actions: Message,
Request Quote/Book, Share, Report (gated for guests via requireAuth).
Done: the profile renders all sections; guest actions bounce to auth and resume; owners see edit affordances.
```

**Model: Sonnet**

```
PROMPT 51: Business profile screen
Depends: 47. Covers: PRO-06.
Build the business profile screen with tabs About/Jobs/Reviews/Team/Documents and the Verified Business badge when applicable.
Done: the business profile renders with all tabs and the verified badge state.
```

**Model: Sonnet**

```
PROMPT 52: Search UI (bar, filters, states)
Depends: 45, 42. Covers: DIS-03, DIS-07, DIS-08, DIS-11.
Build the search UI: search bar with suggestions + category chips; a filter drawer showing the active-filter count on the apply control; filter
state encoded in the URL for shareability/back-nav; skeleton loading; branded empty + "no provider yet in your area" states; list
virtualisation.
Saved/bookmarked workers + searches + recently viewed.
Done: fuzzy debounced search drives results; filters are URL-encoded; empty/no-provider/loading states render; lists virtualise.
```

**Model: Sonnet**

```
PROMPT 53: Guest first-interface (exploration home) [MEETING-NOTES CORE]
Depends: 49, 52, 50, 42. Covers: DIS-01, DIS-02, ACC-01.
Build the FIRST INTERFACE that UNAUTHENTICATED visitors land on, on a single screen, exactly per the meeting notes: (a) a section showing WORKER
PROFILES, (b) a section showing POSTED JOBS, (c) the LOCATION SETTER button (quick-tap current location or open map for a pin), (d) a SEARCH BAR
for searching workers, and (e) a POST-JOB button. Guests can genuinely browse and explore here with NO account: run worker searches, browse
posted
jobs, and open FULL worker profiles (read-only). The purpose of this screen is to show the platform's full potential seamlessly so visitors want
to
create an account. Any state-changing action (post a job, contact, save, apply) triggers requireAuth and resumes after login. Add a map/list
split
with CARTO-clustered markers and a "search this area" control; skeletons + branded empty states. IMPORTANT: this guest view is for
unauthenticated
```

```
users ONLY; once signed in, users are routed to their role-based home (PROMPT 53.1) and never back to this guest view.
Done: a signed-out visitor can search workers, browse posted jobs, and open full profiles with no account, and sees the location setter, search
bar,
and post-job button on one screen; any gated action prompts login then resumes; an authenticated user never sees this guest view.
```

**Model: Sonnet**

```
PROMPT 53.1: Authenticated role-based home and post-login routing
Depends: 16, 50, 53. Covers: ACC-07, DIS-01.
After login (and on every role switch), route users to a ROLE-BASED home for their ACTIVE role instead of the guest exploration view: Worker
home
(incoming job requests, my applications/quotes, active jobs, earnings + Trust Score snapshot, availability quick-toggle); Client home (discover
workers, my posted jobs, received quotes, active bookings); Business home (a summary that the business dashboard in PROMPT 82 later expands).
Reuse
discovery components (worker cards, search, location setter) where useful, but frame and prioritise them per role. The guest exploration view
(PROMPT 53) is reserved for unauthenticated visitors only.
Done: after login a Worker lands on the Worker home, a Client on the Client home, and a Business owner on the Business home; switching role
swaps the
home; no authenticated user is shown the guest view.
```

---

## Phase 4: Transactions

**Model: Haiku** (schema only)

```
PROMPT 54: Jobs schema
Depends: 05, 37. Covers: SRS §6, JOB-05.
Add Prisma models + migration: Job (poster, categoryId, title, description, location geo, budgetMin/Max, urgency, photos, status),
JobApplication (jobId, workerId, status, message). Add the status enum for the lifecycle in SRS §21.3.
Done: migration applies; jobs and applications are modelled with the full status enum.
```

**Model: opusplan** (lifecycle state machine + Tier gate)

```
PROMPT 55: Job posting with Tier 2+ gate and lifecycle
Depends: 54, 25. Covers: JOB-01, JOB-02, JOB-05 (meeting notes: posting limited to higher tier).
Build job creation (REST) GATED to Tier 2+ (ID approved); accounts below Tier 2 receive the exact steps that unlock posting (anti-lure control).
Implement the job lifecycle state machine (Draft -> Open -> Quoted -> Accepted -> In Progress -> Completed -> Released; plus Cancelled,
Disputed,
Archived) with server-side transition guards.
Done: a sub-Tier-2 user is guided to verify instead of posting; a Tier 2+ user creates a job; illegal transitions are rejected.
```

**Model: Sonnet**

```
PROMPT 56: Post-job stepper UI
Depends: 55, 42. Covers: JOB-02.
Build the multi-step post-job stepper (Details -> Location -> Budget -> Review) with a progress indicator, autosave/drafts, photo attachments,
category from the taxonomy, and the Location Setter. Encourage a fixed-price range.
Done: a Tier 2+ user posts a job via the stepper with autosaved drafts; a guest is bounced to auth and resumes the draft.
```

**Model: Sonnet**

```
PROMPT 57: Applications and inviting workers
Depends: 55. Covers: JOB-03, JOB-04.
Build job applications (workers apply with a message/quote) and application status tracking (submitted, under review, shortlisted, not selected,
hired), plus inviting specific workers to a job. Reads via GraphQL, writes via REST.
Done: a worker applies and tracks status; a poster views applicants and can invite specific workers.
```

**Model: Sonnet**

```
PROMPT 58: Job-in-progress view
Depends: 55. Covers: JOB-06, JOB-07.
Build the live Job-in-Progress screen with a status timeline (accepted, on the way, in progress, completed, payment released) and contact
actions;
on completion, confirm and prompt the review + trigger escrow release (wired in P64).
Done: the job screen shows the animated status timeline and completion confirmation.
```

**Model: Sonnet**

```
PROMPT 59: Quotes schema and backend
Depends: 54. Covers: QTE-01, QTE-03.
Add Quote (jobId, workerId, price, durationEst, includes[], message, attachments, status) + migration. Build quote create/send/counter (REST)
with
states Draft/Sent/Viewed/Accepted/Declined/Expired.
Done: a worker sends a quote and it transitions through states; counter-offers are supported.
```

**Model: Sonnet**

```
PROMPT 60: Bookings (accept quote -> booking -> escrow init)
Depends: 59. Covers: QTE-04, PAY-01 (init).
Add Booking (jobId, quoteId, workerId, clientId, agreedPrice, status) + migration. Accepting a quote creates a Booking at the agreed price and
initiates the escrow flow (implemented in P62-P65).
Done: accepting a quote creates a Booking and triggers escrow initiation with the agreed amount.
```

**Model: Sonnet**

```
PROMPT 61: Quote inboxes UI
Depends: 59. Covers: QTE-02, QTE-05.
Build quote inboxes for both sides (Sent / Received / Drafts) with status chips + timestamps; the client quote view shows worker context (Trust
Score, rating, availability) with Accept / Decline / Chat / counter-offer; the worker composer builds itemised quotes.
Done: both parties manage quotes with accurate states; accepting flows into booking + escrow.
```

**Model: opusplan** (payments correctness)

```
PROMPT 62: Campay integration and payment methods
Depends: 05. Covers: PAY-03, PAY-04.
Add Wallet, PaymentMethod, Transaction, EscrowHold models + migration. Integrate Campay for collection + payout in XAF via Mobile Money (MTN,
Orange), card, and bank. Build PaymentMethod management (add/remove/default) with secure storage.
Done: a user adds a payment method; a test Campay collection + payout succeed in sandbox.
```

**Model: Sonnet**

```
PROMPT 63: Wallet backend
Depends: 62. Covers: PAY-05.
Build the wallet backend: available balance, in-escrow amount, Add Money, Withdraw, and a last-transaction summary (reads via GraphQL, writes
via REST).
Done: wallet balance and in-escrow amounts are accurate; add-money and withdraw move funds via Campay.
```

**Model: opusplan** (escrow money state machine)

```
PROMPT 64: Escrow state machine
Depends: 60, 62. Covers: PAY-01, PAY-02, §15.1.
Implement the EscrowHold server-side state machine: Paid -> Accepted -> In Progress -> Completed -> Released, plus Refunded and Disputed. Hold
client
funds from acceptance/payment; release to the worker on client confirmation minus a ~10% fee shown transparently BEFORE payment. Guard every
transition.
Done: a booking funds escrow, moves through states, and releases (minus fee) on client confirmation; illegal transitions are rejected.
```

**Model: opusplan** (webhook idempotency + reconciliation)

```
PROMPT 65: Campay webhooks and reconciliation
Depends: 64. Covers: PAY-06.
Handle Campay webhooks/callbacks to confirm collection + payout; make handlers idempotent (keyed by external ref) and reconcile
escrow/transaction
state after transient failures. Surface explicit failure reasons (declined, insufficient funds, operator timeout with retry). Audit-log every
money event.
Done: duplicate webhooks are safe; state reconciles against Campay; failures return real reasons; every event is audit-logged.
```

**Model: Sonnet**

```
PROMPT 66: Transaction history backend
Depends: 64. Covers: PAY-05.
Build transaction history (payment / escrow_hold / escrow_release / refund / withdrawal / fee) with filters, status, and per-item detail
(GraphQL read).
Done: history lists all transaction types with filters and per-item detail.
```

**Model: Sonnet**

```
PROMPT 67: Wallet, escrow timeline, and transactions UI
Depends: 63, 64, 66. Covers: PAY-02, PAY-05.
Build the wallet UI (balance, in-escrow, add money, withdraw), the animated escrow timeline stepper with a "funds are secure" assurance, the
payment-method manager, and the transaction history view. Show the fee before payment; show explicit failure reasons.
Done: a user funds/tracks escrow visually, manages methods, and reviews transactions; failures show real reasons.
```

**Model: opusplan** (freezes funds, money logic)

```
PROMPT 68: Disputes backend
Depends: 64. Covers: PAY-03, ADM-05 (freeze).
Add Dispute (jobId, openedBy, reason, status open/escalated/resolved, resolution) + migration. Opening a dispute FREEZES the associated escrow
until
resolution; store messages + evidence; resolution actions (release / refund / request info / close) drive the escrow machine and emit
TrustEvents.
Done: opening a dispute freezes escrow; funds cannot move until an admin resolves; the outcome updates escrow + trust.
```

**Model: Sonnet**

```
PROMPT 69: Dispute UI (parties)
Depends: 68. Covers: PAY-03.
Build the dispute detail view for the parties: timeline, messages, evidence upload, and current status.
Done: parties open a dispute, add evidence/messages, and see status; escrow is visibly frozen.
```

---

## Phase 5: Messaging, Safety & Notifications

**Model: Sonnet**

```
PROMPT 70: Messaging schema and backend
Depends: 05, 60. Covers: MSG-01, MSG-02, MSG-04.
Add Conversation (nullable participants for later oversight), Message (kind text/image/voice/location/quote/job/payment, body/mediaKey, status),
```

```
Attachment models + migration. Build conversation/message create + read (writes REST, reads GraphQL) with delivery/read status, block, and
report.
Done: users create job-linked conversations and exchange all message kinds with delivery/read status; block + report work.
```

**Model: Sonnet**

```
PROMPT 71: Real-time WebSocket gateway
Depends: 70. Covers: MSG-01, SRS §13.
Add a NestJS WebSocket gateway for real-time message delivery, typing indicators, and online presence, with a browser WS client in apps/web.
Done: two browsers exchange messages in real time with typing/presence; delivery is live.
```

**Model: Sonnet**

```
PROMPT 72: Messaging UI
Depends: 71, 07. Covers: MSG-01, MSG-02, MSG-03, MSG-05.
Build the inbox (All/Clients/System/Offers filters, search, unread counts, typing/online) and the conversation view with a voice recorder +
waveform, quote/job/payment cards, and inline escrow/job status. Empty states; readable offline for recently synced threads.
Done: users chat with text/voice/cards in real time; escrow/job status shows inline; offline shows last-synced threads.
```

**Model: Sonnet**

```
PROMPT 73: SOS backend
Depends: 05. Covers: SOS-01, SOS-02, SOS-03.
Add SOSAlert (location, status active/resolved/false_alarm) and EmergencyContact models + migration. Build an SOS trigger that shares live
location with the user's emergency contacts and/or the safety process, plus emergency-contact management and an SOS history.
Done: triggering SOS shares location to contacts and records an auditable alert; history lists past alerts with resolution.
```

**Model: Sonnet**

```
PROMPT 74: Live tracking backend (web foreground streaming)
Depends: 71, 40. Covers: SOS-03.
Add LocationPing ingestion over the WebSocket gateway, fed by the browser Geolocation API while the tracking screen is open (web has no reliable
background tracking; stream in the foreground with explicit, revocable consent). Persist pings to PostGIS and fan out to consented viewers with
ETA/distance.
Done: a worker's position streams to a consented client with ETA while the screen is open; consent is revocable; positions persist.
```

**Model: Sonnet**

```
PROMPT 75: Safety UI
Depends: 73, 74. Covers: SOS-01, SOS-02, SOS-03.
Build a press-and-hold SOS control reachable in minimal taps on relevant screens, a Safety Center, the SOS history view, and a consented
live-tracking map for an active job.
Done: SOS is reachable and shares location; the safety center + history render; live tracking shows the worker en route with ETA.
```

**Model: opusplan** (event fan-out + channel design)

```
PROMPT 76: Notifications coverage (domain events -> dispatcher) + admin delivery dashboard
Depends: 11, 26, 64. Covers: NOT-01, SRS §26.1.
Extend the dispatcher (P11) with full event coverage: job match, quote sent/accepted, message received, verification status
change, payment/escrow events, dispute updates, review received, badge unlocked; every template exists in the localized store
(EN/FR) for each active channel and every notification deep-links to its surface. Add the web-push multi-device registry
(DeviceRegistration: userId, platform, token/endpoint unique, deviceName, lastSeen, isActive; deactivate expired
subscriptions on push failure; VAPID keys from env, generated once by pnpm keys). Add an admin delivery dashboard reading
NotificationLog: sent/failed/pending counts by channel, templateKey, and date range.
Done: each event type delivers via push + email per preferences with a log row; a failed or expired push subscription is
deactivated automatically; the admin dashboard shows delivery rates; SMS remains a flag flip away.
```

**Model: Sonnet**

```
PROMPT 77: Web push registration and service-worker handling
Depends: 18, 76. Covers: NOT-02.
Add web push: request permission, register the subscription with the WebPush provider, and handle push events + notification clicks in the
service
worker (deep-linking into the app).
Done: a user opts into web push and receives a clickable push that deep-links into the relevant screen.
```

**Model: Sonnet**

```
PROMPT 78: Notification Center UI and preferences
Depends: 76. Covers: NOT-01, NOT-02.
Build the Notification Center (All/Unread/Important, mark-as-read, detail, deep-link, empty state) and per-channel preferences (in-app / web
push /
email), with SMS shown as "coming soon" and disabled.
Done: users read + filter notifications and set per-channel preferences; SMS is visibly deferred.
```

---

## Phase 6: Reviews, Business & Admin

**Model: Sonnet**

```
PROMPT 79: Reviews schema and backend
Depends: 55, 24. Covers: REV-01, REV-02, REV-03 (meeting notes: reviews with stars + optional text on profile).
Add Review (jobId, authorId, subjectId, rating 1-5, text, photos) + migration. Allow a review only from a COMPLETED job's two participants
```

```
(prevents fabrication). A new review feeds the Marketplace Trust sub-score (TrustEvent). Support reporting.
Done: only completed-job participants can review; a new review updates the marketplace sub-score; reporting works.
```

**Model: Sonnet**

```
PROMPT 80: Reviews UI
Depends: 79, 50. Covers: REV-01, REV-02.
Build the review submit flow (stars + optional text + optional photos) and the profile reviews summary (average, distribution bars, count) +
full
list, shown on the worker's profile.
Done: a client leaves a star + text review after a completed job; it appears in the worker's profile summary and list.
```

**Model: opusplan** (access-level enforcement)

```
PROMPT 81: Business and team backend
Depends: 22, 63. Covers: BIZ-01, BIZ-02, BIZ-03, BIZ-04.
Add TeamMember (role, accessLevel full/manage-jobs/payments/view-only, status) + migration. Build enhanced Business Verification (RCCM) granting
a
Verified Business badge + Tier-3-gated corporate eligibility, team-member management + invitations, a company wallet, business analytics, and
team/bulk job posting.
Done: a business verifies, invites scoped team members, posts a team job, and sees analytics + a company wallet; access levels enforced server-
side.
```

**Model: Sonnet**

```
PROMPT 82: Business dashboard and team UI
Depends: 81. Covers: BIZ-01, BIZ-03.
Build the business dashboard (active posts, applications, in-progress/completed, spend, recent payments) and the team-management UI (roles,
access
levels, invitations, status).
Done: a business owner manages the team with scoped access and sees the dashboard metrics.
```

**Model: Sonnet**

```
PROMPT 83: Admin dashboard, analytics, and reports
Depends: 15. Covers: ADM-01, ADM-07.
Build the admin dashboard (dark, dense): platform KPIs (total users, verified workers, jobs completed, revenue with deltas), performance charts,
jobs-by-category, escrow/disputes overviews, SOS alerts, system health; plus Analytics & Reports (user growth, jobs over time, jobs by
category/location, revenue by source, trust-score distribution, top performers) with export. RBAC: admin.
Done: an admin sees live KPIs + charts and exports a report; all admin-gated.
```

**Model: opusplan** (trust-score integrity)

```
PROMPT 84: Admin verification queue and Trust Score adjustment
Depends: 29, 26. Covers: ADM-03, ADM-04.
Build the admin verification queue (All/ID/Business/Field) with a secure document viewer and approve/reject/request-more-info (reasons feed the
friendly-retry UX), plus the ability to adjust a Trust Score in exceptional cases (fraud penalty / Field-Verified bonus) via an explicit
TrustEvent.
Done: an admin clears the queue and can apply an auditable Trust Score adjustment via an event.
```

**Model: opusplan** (drives escrow)

```
PROMPT 85: Admin dispute-resolution workspace
Depends: 68. Covers: ADM-05.
Build the admin dispute workspace: details, messages, evidence, history, and resolution actions (release payment / refund client / request info
/
close) that drive the escrow state machine and emit TrustEvents.
Done: an admin resolves a dispute; the chosen action moves escrow and updates trust; the action is audit-logged.
```

**Model: Sonnet**

```
PROMPT 86: Admin fraud & risk monitor
Depends: 83. Covers: ADM-06.
Build the Fraud & Risk monitor (high-risk users, suspicious jobs, blocked accounts, risk alerts) and a platform activity heatmap fed by
CARTO/PostGIS.
Done: an admin sees risk signals and a live activity heatmap; can block/flag accounts.
```

**Model: Sonnet**

```
PROMPT 87: Admin user/agent management and agent invite
Depends: 15. Covers: ADM-02 (users), ADM-06, meeting-notes admin invites agents.
Build management for Users/Workers/Businesses/Agents (search, view, edit, suspend) and the ability for admins to INVITE agents (create + onboard
agent accounts). RBAC: admin.
Done: an admin manages all account types and can invite a new agent who can then onboard.
```

**Model: Sonnet**

```
PROMPT 88: Admin taxonomy, announcements, audit, RBAC, settings
Depends: 43, 83. Covers: ADM-02, ADM-08.
Round out admin ops: service-category management surface (on top of P43), Announcements, an Audit Log viewer, Roles & Permissions (RBAC config),
Integrations, and System Settings.
Done: an admin edits categories, posts announcements, browses audit logs, configures roles, and manages settings.
```

---

# Phase 7: PWA Hardening & Marketing

**Model: opusplan** (offline sync + conflict handling)

```
PROMPT 89: Offline browsing and write queue (PWA)
Depends: 18. Covers: SYS-02, NFR-PWA.
Give the installable PWA real offline functionality as planned: app-shell caching + stale-while-revalidate so previously viewed worker profiles, job
posts, and discovery content stay browsable offline, and recently synced messages stay readable offline, all behind a clear offline indicator. Persist
the web Apollo cache (IndexedDB) for instant offline-first reads. Queue write actions taken offline (drafts, edits) and sync them on reconnect
(conflict-safe, last-write-wins for simple fields), reconciling the Apollo cache.
Done: with the network off, the user can browse previously viewed profiles/jobs and read synced messages behind an offline indicator; an action
drafted offline syncs on reconnect.
```

**Model: Haiku** (i18n wiring)

```
PROMPT 90: Internationalisation (EN/FR)
Depends: 06. Covers: SYS-03, NFR-I18N.
Wire full EN/FR i18n via next-intl, including dates, numbers, and XAF currency formatting. Audit the app for any hardcoded user-facing strings
and move them to messages.
Done: the app toggles fully EN <-> FR; currency/date/number formatting is correct; no hardcoded strings remain.
```

**Model: Haiku** (static states)

```
PROMPT 91: System states
Depends: 04. Covers: SYS-04.
Implement consistent system states across the app: 404, 500, offline/no-connection, maintenance, loading (skeletons, not spinners), empty, and
success.
Done: each state has a branded screen/component and is wired where it applies.
```

**Model: Sonnet**

```
PROMPT 92: Marketing landing page (signature)
Depends: 08, 33. Covers: SYS-05, §20.6.
Build the signature landing (marketing motion profile, excluded from the service-worker scope): hero with worker/client portraits + floating trust
badge + animated Trust Score; social-proof count-ups; a Trust Engine section visualising score progression on scroll; horizontal portfolio with
before/after; escrow-explained pricing; a testimonial with a voice waveform; a single dark CTA routing into the app.
Done: the landing tells the trust story on scroll, is not controlled by the app service worker, and its CTAs enter the app.
```

**Model: Sonnet**

```
PROMPT 92.1: Database-driven FAQ + testimonials (admin CRUD) and landing hygiene
Depends: 92, 43. Covers: SRS §26.2, §26.6.
Make marketing content database-driven, never hardcoded: FAQ (id, question, answer markdown, category?, order, isPublished)
and Testimonial (author, role, avatar, content, rating decimal, order, visible) models + migration, admin CRUD with reorder
and publish toggles, and public GraphQL reads. Frontend: FAQ as a smooth animated accordion (animated height, rotating
chevron, reusable AccordionItem primitive); testimonials as an infinite marquee (auto-scroll, pause on hover, drag support,
edge gradient fade, fixed card width, line-clamped text) seeded with at least 8 entries; star ratings render decimals with
half-stars (4.3 -> 4 full + 1 half), aria-labelled, computed server-side. Landing hygiene: no internal workflow links
(admin/agent logins) anywhere on the conversion surface; enforce the 60/30/10 colour ratio (accent visible on every primary
CTA) and document it in BRAND_GUIDELINES.md.
Done: FAQ and testimonials are editable from admin and render with the specified motion; a 4.3 rating shows correctly
rounded half-stars; no admin/agent entry points exist on marketing pages.
```

**Model: Sonnet**

```
PROMPT 93.1: SEO hygiene, legal pages, and analytics events
Depends: 93. Covers: SRS §26.4, §26.6.
Apply the SEO hygiene rules: noindex (meta + robots.txt) for paginated variants beyond page 1, filtered discovery URLs,
search results, and account/auth/preview routes; server-derived canonicals stripped of query params (filtered discovery ->
clean base, worker/job detail -> clean slug, page 1 canonical for pagination; never from window.location). Add a
route-metadata integrity check (script or CI step) asserting every public route exports title, description, canonical, and an
OG image. Ship static legal pages (Terms, Privacy, Refund/Dispute policy) linked in the footer. Add a typed analytics event
layer tracking the conversion funnel (landing CTA -> sign-up -> OTP verified -> verification steps -> first job/quote ->
first escrow) with event names in one const.
Done: filtered/paginated URLs carry noindex + clean canonicals; the metadata check fails on a route missing metadata; legal
pages render; funnel events fire with typed names.
```

**Model: Haiku** (SEO metadata)

```
PROMPT 93: SEO
Depends: 92. Covers: SYS-05.
Add SEO to the marketing site: metadata, canonical, OG/Twitter, JSON-LD structured data, sitemap, robots; render public pages static/ISR.
Done: Lighthouse SEO passes on the landing; structured data validates; sitemap/robots are served.
```

**Model: Sonnet**

```
PROMPT 94: Accessibility pass
Depends: across app. Covers: NFR-A11Y.
Do a WCAG 2.1 AA pass: keyboard operability, focus management/traps in overlays, ARIA roles, sufficient contrast, and prefers-reduced-motion across
all interactive surfaces.
Done: representative pages pass an a11y audit; overlays trap focus; reduced-motion is honoured everywhere.
```

**Model: Sonnet**

```
PROMPT 95: Performance budgets
Depends: across app. Covers: NFR-PERF.
Meet the budgets on a mid-range phone browser over 3-4G: LCP < 2.5s, CLS < 0.1, INP < 100ms. Add responsive images, lazy loading, code-splitting,
and confirm list virtualisation on long lists.
Done: Lighthouse performance meets the budgets on the landing and a representative app page.
```

**Model: Haiku** (install polish)

```
PROMPT 96: PWA install polish
Depends: 18, 92. Covers: SYS-01.
Polish installability: a custom install prompt, complete icon set, splash screens, and correct theming, so the PWA installs cleanly on phone and desktop.
Done: the install prompt appears at the right moment and the installed PWA launches with correct icons/splash/theme.
```

---

## Appendix A: Definition of Done (every prompt/PR)

- ☐ Typechecks and lints pass; new logic has unit + integration tests.
- ☐ Reads via GraphQL/Apollo; writes via REST/fetch (dual-API rule upheld).
- ☐ RBAC enforced server-side for every write and privileged read; active-role respected.
- ☐ Inputs validated (Zod/DTO) before the DB; public + auth endpoints rate-limited.
- ☐ Indexes added for any new frequent query path; no N+1 (select/include + pagination).
- ☐ Loading (skeleton), empty (branded), and error (inline, with requestId) states present.
- ☐ EN/FR strings (no hardcoded copy); a11y (keyboard, focus, contrast, reduced-motion).
- ☐ Money/verification actions audit-logged; migrations deploy-time only; no secrets committed.
- ☐ Notifications/OTP go through the channel service (never a hardcoded SMS call).
- ☐ PR references the SRS requirement IDs covered and the PROMPT number.

## Appendix B: Reusable atomic-prompt template

**Model: Haiku | Sonnet | opusplan** (choose per the routing guide above)

```
PROMPT NN: <single focused task>
Depends: <prompt numbers>. Covers: <SRS requirement IDs / sections>.
<One task only. State the backend piece (Prisma model + migration if new; NestJS service/resolver/controller; validation; guard; events) OR the frontend piece (one screen/component; TMDS; states; motion; EN/FR), not a bundle of both unless they are inseparable.>
Done: <one or two observable acceptance criteria, plus the Definition of Done>.
```

## Appendix C: How to run the queue

1. Paste STEP 0 into `CLAUDE.md` once.
2. Run prompts in order. Each says what it depends on by number; do not start one until its dependencies are green.
3. One prompt = one branch (`feat/NN-short-name`) = one PR. Finish with the prompt's *Done* line + Appendix A, then merge.
4. If Claude Code proposes native mobile, Expo, or breaking the dual-API rule, stop it and re-anchor to `CLAUDE.md`. This platform is web-only.

**Web-only reminder:** there is no native app. "Mobile" always means the responsive, installable PWA in a phone browser. Real-time location is foreground browser geolocation streamed over WebSocket while the screen is open (P74), not background tracking.

*End of queue.*